File Maintenance Tool

A Windows-first, unattended file maintenance utility for scheduled cleanup jobs. The core cleanup logic is isolated from OS-specific setup, notification, path, and disk-space behavior through the `internal/platform` abstraction.

The tool can:

- Scan configured file and folder paths.
- Identify files older than the configured retention period.
- Back up eligible files before deletion when backup is enabled for that path.
- Validate backup destination capacity once per queued batch instead of once per file.
- Delete eligible files directly when backup is intentionally disabled for that path.
- Write operational, error, and count logs with retention cleanup.
- Open a Windows setup wizard by default when the executable is launched without `-run`.
- Run backup/delete maintenance only when `-run` is passed or when Save & Run is selected in the setup wizard.
- Print build metadata with the `-version` flag.

Current Status

This project is currently optimized for Windows scheduled execution.

Area	Current status
Windows setup wizard	Implemented with embedded PowerShell and Windows Forms; opens by default when `-run` is not passed.
Windows notifications	Implemented through PowerShell / Windows Forms message boxes.
Windows backup space validation	Implemented through `GetDiskFreeSpaceEx`; checked once per queued worker batch.
Linux/macOS setup mode	No GUI setup wizard yet; default setup mode exits safely with a setup-not-implemented error.
Linux/macOS backup space validation	Not implemented yet. Backup-enabled runs on those platforms will fail when batch space validation runs.
GitHub Actions build matrix	Windows amd64, Linux amd64, macOS amd64, and macOS arm64.

Current Default Layout

The application currently uses portable defaults beside the executable:

```
fileMaintenance.exe
config/
  config.ini
  logging.json        # optional
logs/
  maintenance_YYYY-MM-DD.log
  errors_YYYY-MM-DD.log
  count_YYYY-MM-DD.log
```

This is intentional. The platform package still supports OS-conventional directories such as AppData on Windows, but `cmd/main/main.go` currently defaults to `<exe>/config` and `<exe>/logs` to make Task Scheduler deployments and portable runs easier to reason about.

Override paths with:

```
fileMaintenance.exe -config-dir "C:\path\to\config" -log-dir "C:\path\to\logs"
```

First-Time Setup and Run Modes

The executable is setup-first by default. This prevents backup/delete maintenance from starting accidentally when the program is double-clicked.

`fileMaintenance.exe`

On Windows, this opens the embedded PowerShell setup wizard. The wizard writes:

`<config-dir>/config.ini`

The wizard has three exit options:

Option	Behavior
Cancel	Close setup and do not run maintenance.
Save & Close	Save `config.ini` and exit without running maintenance.
Save & Run	Save `config.ini`, close setup, then immediately run backup/delete maintenance.

Background maintenance is intentionally gated behind `-run`:

`fileMaintenance.exe -run`

When `-run` is used, the application requires an existing `config.ini`. It does not open the GUI during background/scheduled execution.

On Linux and macOS, no GUI setup wizard is currently provided. Launching without `-run` exits safely with a setup-not-implemented message.

How It Works

1. **Startup**
   - Resolve the active OS platform implementation.
   - Resolve the executable directory.
   - Set portable default paths for `config/` and `logs/`.
   - Parse CLI flags.
   - Initialize logging.
2. **Mode Selection**
   - If `-run` is not passed, open setup/configuration mode.
   - On Windows, the setup wizard can Cancel, Save & Close, or Save & Run.
   - If `-run` is passed, require an existing `config.ini` and do not open the GUI.
3. **Configuration Loading**
   - Read `config.ini`.
   - Parse the maintenance plan from `[backup]` and `[paths]`.
   - Parse runtime settings from optional `[settings]` and `[advanced]` sections.
   - Resolve runtime precedence as defaults → `config.ini` → explicitly passed CLI flags.
4. **Safety Checks**
   - Determine whether any configured path has backup enabled.
   - If backup is required, validate the backup destination before deleting anything.
   - Show a platform-specific critical notification if the backup path is inaccessible.
5. **Maintenance Worker**
   - Scan configured paths using bounded walkers.
   - Collect candidate jobs into an in-memory batch up to `queue-size`.

- When the batch is full, or when walking finishes, total the backup-enabled file sizes in that batch.
- Validate backup destination capacity once for the full batch before copying or deleting.
- Process the full batch through one serialized processor before accepting the next batch.
- Copy before delete when backup is enabled.
- Delete directly only when backup is disabled for that path.
- Track per-path delete counts.

6. **Cleanup and Exit**
   - Prune old logs according to log retention.
   - Exit with an error code when fatal configuration or worker errors occur.

Execution Flow

The high-level Mermaid source is maintained in:

`docs/diagrams/execution-flow-high-level.md`

The detailed execution flow is maintained in:

`docs/diagrams/execution-flow.md`

The execution flow uses concurrent path discovery, batch-level backup-space validation, and serialized file operations. While a full batch is being checked and processed, walkers are blocked from adding more jobs, which prevents unbounded queue growth and avoids repeated per-file destination-space checks.

Command-Line Flags

Mode

Flag	Default	Description
`-run`	`false`	Run the background backup/delete maintenance process. Required for scheduled maintenance.
`-setup`	`false`	Open the setup/configuration UI. This is also the default behavior when `-run` is not passed.

Retention and Logging

Flag	Default	Description
`-days`	7	Only files older than this many days are eligible. `0` effectively selects files older than the current instant.
`-log-retention`	30	Number of days to keep log files.
`-no-logs`	`false`	Disable file logging and write to stdout/stderr.
`-no-backup`	`false`	Disable all backups for this run and delete eligible files directly. Requires intentional use with `-run`.
`-version`	`false`	Print version metadata and exit.

Paths

Flag	Default	Description
`-config-dir`	`<exe>/config`	Directory containing `config.ini` and optional `logging.json`.
`-log-dir`	`<exe>/logs`	Directory where log files are written.

Resource Controls

Flag	Default	Description
`-walkers`	1	Number of concurrent path walkers.
`-queue-size`	300	Maximum jobs collected in one worker batch before destination space is checked and the batch is processed.
`-max-files`	0	Maximum jobs to handle in one run. `0` means unlimited.
`-max-runtime`	30m	Maximum run duration. `0` means unlimited. CLI values use Go duration strings such as `55m`, `1h`, or `30s`.
`-cooldown`	0	Delay after each processed job. Useful for SMB/network pacing. CLI values use Go duration strings such as `50ms` or `1s`.
`-retries`	2	Number of backup copy retries.

Important: backup/delete maintenance only runs when `-run` is passed or when Save & Run is selected in the Windows setup wizard. Runtime precedence is defaults → `config.ini` → explicitly passed CLI flags.

Configuration

`config/config.ini`

Required sections:

```
[backup]
path=D:\backups

[paths]
C:\Temp\OldFiles, yes
C:\Temp\ToDelete, no
```

Optional sections:

```
[settings]
days=7
log-retention=30

[advanced]
walkers=1
queue-size=300
retries=2
cooldown=50ms
max-files=0
max-runtime=55m
no-backup=false
```

`config.ini` now supports the same duration style as the CLI for runtime values such as `cooldown=50ms` and `max-runtime=55m`. Plain numeric duration values are still accepted for backward compatibility and are interpreted as milliseconds.

Explicit zero values are valid in `config.ini`. For example, `days=0`, `max-files=0`, and `max-runtime=0` are treated as intentional configured values rather than ignored defaults.

[backup]

Key	Description
path	Backup destination root path. Can be local or network/SMB.

[paths]

Each standalone line is a file or folder path followed by optional backup behavior:

`path, yes|no`

Value	Meaning
yes	Back up eligible files before deletion.
no	Delete eligible files without backup. Use carefully.
omitted	Backup defaults to enabled.

Supported path types:

Type	Example	Behavior
Folder	`C:\Temp\OldFiles, yes`	Recursively evaluates files inside the folder.
File	`C:\Logs\old.log, no`	Evaluates that file directly.

Comments and blank lines are ignored. Lines beginning with `;` or `#` are treated as comments.

config/logging.json

Optional logging configuration:

```
{
    "DEBUG": false,
    "COUNT": true,
    "INFO": true,
    "WARN": true,
    "ERROR": true,
    "SUCCESS": true,
    "FATAL": true
}
```

Unknown log levels default to enabled.

Backup Layout

Backups are written under a run-date folder:

`<backupRoot>/<DDMmmYY>/<folder-name>/<relative-path>/<filename>`

Example:

```
Source:
C:\Data\Images\2024\Camera\IMG001.jpg

Backup:
D:\backups\30Jan26\Images\2024\Camera\IMG001.jpg
```

The active backup path builder preserves the source folder structure under the dated backup folder. The worker supplies files discovered from the configured source root, and the copy layer creates the destination folders as needed.

Backup Space Validation

When backup is enabled for a path, each queued `FileJob` carries the source file size. The worker collects jobs into an in-memory batch with a maximum size controlled by `queue-size`.

The backup-space check now happens once per batch:

1. Walkers enqueue eligible files until the batch reaches `queue-size`, or until no more candidate files remain.
2. The worker totals the size of backup-enabled files in the current batch. Delete-only jobs do not increase the required backup bytes.
3. The worker calls `AvailableBytes(backupRoot)` once and compares the available destination space with the full batch requirement.
4. If enough space is available, the worker processes the complete batch serially before accepting the next batch.
5. If space is insufficient, the worker cancels the run before the batch is copied or deleted, so the source files remain in place.

This replaces the previous per-file destination-space check. If another process consumes backup-destination space after the batch check, an individual copy can still fail; in that case, the source file is not deleted because deletion only occurs after a successful backup copy.

Current platform limitation: this validation is implemented for Windows. Linux and macOS currently return a `disk space check not implemented for this platform` error when backup-enabled work reaches batch validation.

Logging

Default file logs:

```
logs/maintenance_YYYY-MM-DD.log   # all enabled levels
logs/errors_YYYY-MM-DD.log        # ERROR/FATAL-focused output
logs/count_YYYY-MM-DD.log         # summary counts
```

Per-path delete counts are logged after processing completes so totals reflect actual successful deletions.

OS-Specific Platform Abstraction

The platform layer lives under `internal/platform`:

```
internal/platform/
  platform.go
  current_windows.go
  current_linux.go
  current_darwin.go
  windows/
  linux/
  macos/
```

The core application depends on the `Platform` interface instead of importing OS-specific packages directly.

Current responsibilities:

- `ShowCritical(title, message string)`
- `DefaultConfigDir(appName string) (string, error)`
- `DefaultLogDir(appName string) (string, error)`
- `EnsureConfig(configDir string, exeDir string) (bool, error)`
- `AvailableBytes(path string) (uint64, error)`

Windows provides the setup wizard, Save & Close / Save & Run actions, and disk-space implementation. Linux and macOS currently do not implement the setup wizard and do not implement backup destination free-space checks yet.

Windows Task Scheduler Setup

Recommended Task Scheduler action fields for the current Windows deployment:

Program/script:

`powershell.exe`

Add arguments (shown wrapped; paste as one line in Task Scheduler):

```
-NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden
-Command "& 'C:\FileCopy\fileMaintenance\fileMaintenance.exe' -run -days 0
-walkers 1 -max-runtime 55m -cooldown 50ms"
```

Start in:

`C:\FileCopy\FileMaintenance`

Equivalent PowerShell command, split for readability:

```powershell
$exe = "C:\FileCopy\fileMaintenance\fileMaintenance.exe"
$args = "-run -days 0 -walkers 1 -max-runtime 55m -cooldown 50ms"

powershell.exe -NoProfile `
  -ExecutionPolicy Bypass `
  -WindowStyle Hidden `
  -Command "& '$exe' $args"
```

Recommended task options:

- Run whether user is logged on or not, if the configured paths and backup destination are available to that account.
- Run as soon as possible after a missed start.
- Stop the task if it runs longer than expected.
- Use `-run` for scheduled/background maintenance. Explicit CLI runtime flags override matching values in `config.ini`.

For the current scheduled command:

- `-days 0` makes almost all files older than the current instant eligible.
- `-walkers 1` keeps folder discovery serialized.
- `-max-runtime 55m` caps the run at 55 minutes and overrides the matching `config.ini` value because the flag is explicit.
- `-cooldown 50ms` adds a short pause after each processed job and overrides the matching `config.ini` value because the flag is explicit.

GitHub Actions Release Build

The GitHub Actions workflow is located at:

`.github/workflows/build.yml`

It currently:

- Runs on pull requests to `main`.
- Runs on pushed tags matching `v*`.
- Supports manual `workflow_dispatch` runs.
- Runs `go test ./...` before building each OS/architecture target.
- Builds platform-specific archives:
  - Windows amd64: `.zip`
  - Linux amd64: `.tar.gz`
  - macOS amd64: `.tar.gz`
  - macOS arm64: `.tar.gz`
- Injects version metadata through linker flags:
  - `Version`
  - `Commit`
  - `BuildDate`
- Creates a GitHub release for tag pushes when release assets are available.

Check build metadata locally or from a release binary with:

```
.\fileMaintenance.exe -version
```

Safety Guarantees

- No deletion occurs if backup is enabled and the backup root is inaccessible.
- No batch is copied or deleted if the total backup-enabled size of that batch exceeds available backup destination space.
- No deletion occurs if backup copy fails.
- File operations are serialized to reduce network and disk contention.
- Resource controls prevent unbounded walking or job queue growth.
- Critical backup-location failures trigger platform-specific user notification.

Development and Testing

The project currently targets the Go version declared in `go.mod`.

Run tests:

```
go test ./...
```

Run the worker batch-space tests only:

```
go test ./internal/maintenance `
  -run "BackupSpaceCheckedOncePerBatch|InsufficientBackupSpaceForBatch" `
  -v `
  -count=1
```

Build:

```
.\build.ps1 build
```

Run locally:

```
.\fileMaintenance.exe -run -days 0
```

Run through the helper script:

```
.\build.ps1 run -Days 0
```

The `build.ps1 smoke` target now checks for `config/config.ini` and runs the executable with `-run -no-logs -days 0`.

License

Internal / private use.